COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	July 17, 2026	**DATE SUBMITTED:**	July 22, 2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr, Jimlord Quejado

ACTIVITY NO. 1 **Algorithm, Flowchart and Pseudocode**

1. PROCEDURE OUTPUT

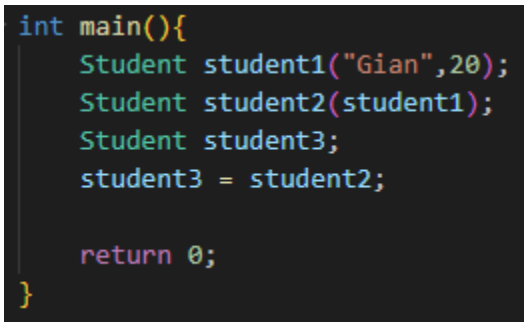
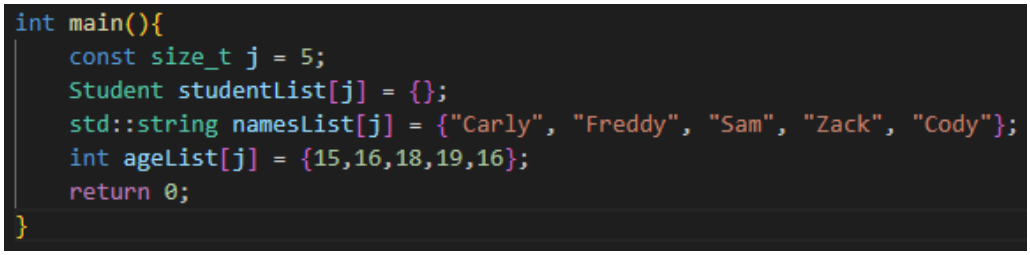
Screenshot	
Observation	First, the members of the student class initialized the value of student one which we can see since the program prints “Constructor called”. Meaning we successfully created a new object. After that, the value of student1 was successfully copied by student2 which gave the two variables similar values. We can see the copy constructor called as proof. Then, a new object was created by the user with different data but later on got replaced by the same data in student1 and 2 variables. After the variables were used, they were successfully deleted by the program by using the destructor function.

Table 2-1. Initial Driver Program

Screenshot	

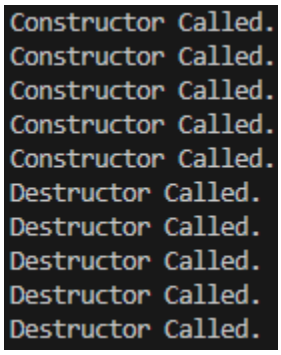
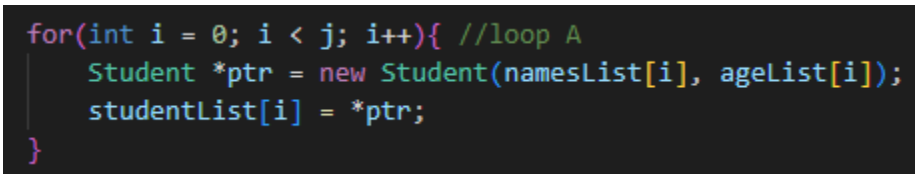
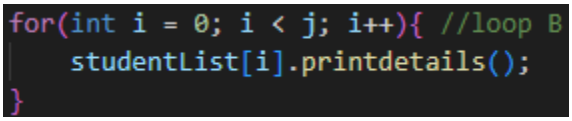
	
Observation	Based on the visualizer, the data of the array was all set into a single value that was created by the user. After all 5 members of the arrays were set into the same value, they are then replaced by the new values in the arrays and were created individually which is why we get to see 5 “constructor called” messages in the output. After the program used these constructors, they were instantly deleted.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	The data from the previous arrays were replacing the default values or what we call the “placeholder values” in the main arrays by looping it one at a time, changing the ages first and then the names of the students.
Loop B	
Observation	The data from the newly created arrays are shown to the users by printing it in order.

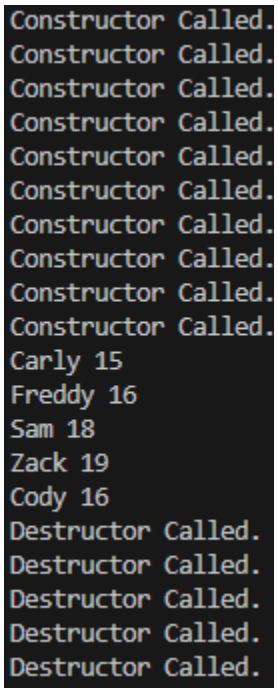
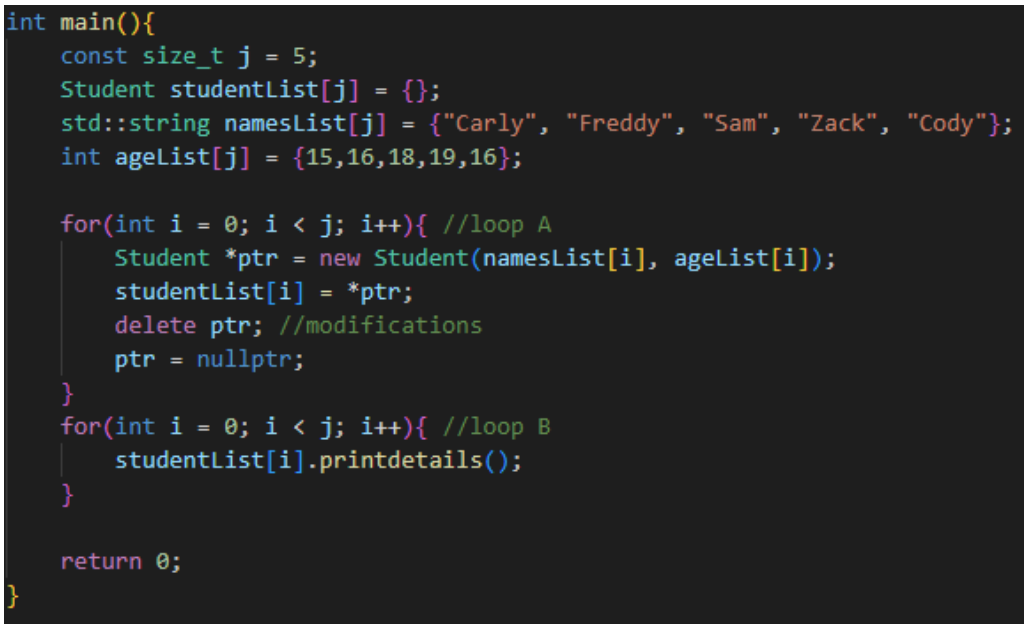
Output	
Observation	As you can see, after creating a new array, the system calls the constructor five times but that is only for the placeholder value. After the looping, it called the constructor for another 5 times signaling that the values have been changed. After that, it shows the details of the array and then destroys the details after the program.

Table 2-3. Final Driver Program

Modifications	

```
int main(){
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15,16,18,19,16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
        delete ptr; //modifications
        ptr = nullptr;
    }
    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printdetails();
    }

    return 0;
}
```

Observation	Basically, all the delete ptr is doing is removing the memory that was used by the system. The purpose of the nullptr is just a safety precaution so that if a pointer is deleted, it will automatically point to null.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List.

You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

```cpp
#include <iostream>
#include <string>  // For Strings
#include <iomanip> // Table design

// FRUIT CLASS
class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    // CONSTRUCTOR
    Fruit(const std::string& n = " ", double p = 0.0, int q = 0)
        : name(n), price(p), quantity(q) {}

    // DESTRUCTOR
    ~Fruit() {
        std::cout << "Fruit Destructor Called: " << name << std::endl;
    }

    // COPY CONSTRUCTOR
    Fruit(const Fruit& other)
        : name(other.name), price(other.price), quantity(other.quantity) {}
```

Image 1

```
    // COPY ASSIGNMENT
    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    double totalSum() const {
        return price * quantity;
    }

    void receipt() const {
        std::cout << std::left
                  << std::setw(15) << "Fruit"
                  << std::setw(15) << name
                  << std::setw(10) << price
                  << std::setw(10) << quantity
                  << std::setw(10) << totalSum()
                  << std::endl;
    }
};
```

Image 2

```
// VEGETABLE CLASS
class Vegetable {
private:
    std::string name;
    double price;
    int quantity;

public:
    // CONSTRUCTOR (Using C++ Member Initializer List)
    Vegetable(const std::string& n = " ", double p = 0.0, int q = 0)
        : name(n), price(p), quantity(q) {}

    // DESTRUCTOR
    ~Vegetable() {
        std::cout << "Vegetable Destructor Called: " << name << std::endl;
    }

    // COPY CONSTRUCTOR
    Vegetable(const Vegetable& other)
        : name(other.name), price(other.price), quantity(other.quantity) {}

    // COPY ASSIGNMENT
    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }
```

Image 3

```
    double totalSum() const {
        return price * quantity;
    }

    void receipt() const {
        std::cout << std::left
                  << std::setw(15) << "Vegetables"
                  << std::setw(15) << name
                  << std::setw(10) << price
                  << std::setw(10) << quantity
                  << std::setw(10) << totalSum()
                  << std::endl;
    }
};

// Problem 3 : TotalSum
double totalSum(Fruit* fruits[], int fruitCount,
                Vegetable* vegetables[], int vegetableCount);

// THE MAIN PROGRAM
int main() {

    // Problem 2: Creating a GroceryList Array
    Fruit* fruitsTaken[2] = { nullptr, nullptr };
    Vegetable* vegetablesTaken[2] = { nullptr, nullptr };

    fruitsTaken[0] = new Fruit("Apple", 10, 7);
    fruitsTaken[1] = new Fruit("Banana", 10, 8);

    vegetablesTaken[0] = new Vegetable("Broccoli", 60, 12);
    vegetablesTaken[1] = new Vegetable("Lettuce", 50, 10);
```

Image 4

```
    std::cout << "============================================================\n";
    std::cout << "               JENNA'S GROCERY RECEIPT\n";
    std::cout << "============================================================\n";

    std::cout << std::left
              << std::setw(15) << "Type"
              << std::setw(15) << "Item"
              << std::setw(10) << "Price"
              << std::setw(10) << "Quantity"
              << std::setw(10) << "Total"
              << std::endl;
    std::cout << "------------------------------------------------------------\n";

    std::cout << "Fruits:" << std::endl;
    for (int i = 0; i < 2; ++i) {
        if (fruitsTaken[i] != nullptr) {
            fruitsTaken[i]->receipt();
        }
    }

    std::cout << "Vegetables:" << std::endl;
    for (int i = 0; i < 2; ++i) {
        if (vegetablesTaken[i] != nullptr) {
            vegetablesTaken[i]->receipt();
        }
    }

    std::cout << "\nTotal Amount in Pesos: "
              << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
              << std::endl;
```

Image 5

```
std::cout << "Fruits:" << std::endl;
for (int i = 0; i < 2; ++i) {
    if (fruitsTaken[i] != nullptr) {
        fruitsTaken[i]->receipt();
    }
}

std::cout << "Vegetables:" << std::endl;
for (int i = 0; i < 2; ++i) {
    if (vegetablesTaken[i] != nullptr) {
        vegetablesTaken[i]->receipt();
    }
}

std::cout << "\nTotal Amount in Pesos: "
          << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
          << std::endl;

// Problem 4 : Deleting Lettuce from Jenna Receipt
char answer;
std::cout << "\nProceed to Delete Lettuce? (Y/N): ";
std::cin >> answer;

if (answer == 'Y' || answer == 'y') {
    std::cout << "\n========== POINTER LIST BEFORE DELETION ==========\n";
    std::cout << "Apple: " << fruitsTaken[0] << std::endl;
    std::cout << "Banana: " << fruitsTaken[1] << std::endl;
    std::cout << "Broccoli: " << vegetablesTaken[0] << std::endl;
    std::cout << "Lettuce: " << vegetablesTaken[1] << std::endl;
    std::cout << "================================================\n";
```

Image 6

```
    std::cout << "\nMemory successfully deallocated.\n";

    std::cout << "\n========== POINTER LIST AFTER DELETION ==========\n";
    std::cout << "Apple: " << fruitsTaken[0] << std::endl;
    std::cout << "Banana: " << fruitsTaken[1] << std::endl;
    std::cout << "Broccoli: " << vegetablesTaken[0] << std::endl;
    std::cout << "Lettuce: " << vegetablesTaken[1] << std::endl;
    std::cout << "================================================\n";

    if (vegetablesTaken[1] == nullptr) {
        std::cout << "\nLettuce successfully deleted.\n";
        std::cout << "Lettuce pointer is now nullptr.\n";
    }
} else {
    std::cout << "\nDeletion cancelled.\n";
}

std::cout << "\nPress Enter to Continue...";
std::cin.ignore();
std::cin.get();
```

Image 7

```
std::cout << "\n============================================================\n";
std::cout << "              UPDATED GROCERY RECEIPT\n";
std::cout << "============================================================\n";

std::cout << "Fruits:" << std::endl;
for (int i = 0; i < 2; ++i) {
    if (fruitsTaken[i] != nullptr) {
        fruitsTaken[i]->receipt();
    }
}

std::cout << "\nVegetables:" << std::endl;
for (int i = 0; i < 2; ++i) {
    if (vegetablesTaken[i] != nullptr) {
        vegetablesTaken[i]->receipt();
    }
}

std::cout << "\nNew Total Amount in Pesos: "
          << totalSum(fruitsTaken, 2, vegetablesTaken, 2)
          << std::endl;

// Cleaning up remaining memory to prevent memory leaks
std::cout << "\n========== POINTER LIST BEFORE FINAL CLEANUP ==========\n";
std::cout << "Apple: " << fruitsTaken[0] << std::endl;
std::cout << "Banana: " << fruitsTaken[1] << std::endl;
std::cout << "Broccoli: " << vegetablesTaken[0] << std::endl;
std::cout << "Lettuce: " << vegetablesTaken[1] << std::endl;
std::cout << "============================================================\n";
```

Image 8

```
    std::cout << "\nCleaning up remaining memory\n";

    // Delete Apple and Banana
    for (int i = 0; i < 2; ++i) {
        delete fruitsTaken[i];
        fruitsTaken[i] = nullptr;
    }

    // Delete Broccoli
    delete vegetablesTaken[0];
    vegetablesTaken[0] = nullptr;

    std::cout << "\nMemory cleaned up successfully.\n";

    std::cout << "\n========== POINTER LIST AFTER FINAL CLEANUP ==========\n";
    std::cout << "Apple: " << fruitsTaken[0] << std::endl;
    std::cout << "Banana: " << fruitsTaken[1] << std::endl;
    std::cout << "Broccoli: " << vegetablesTaken[0] << std::endl;
    std::cout << "Lettuce: " << vegetablesTaken[1] << std::endl;
    std::cout << "============================================================\n";

    if (fruitsTaken[0] == nullptr &&
        fruitsTaken[1] == nullptr &&
        vegetablesTaken[0] == nullptr &&
        vegetablesTaken[1] == nullptr) {
        std::cout << "\nAll allocated memory has been deallocated successfully.\n";
        std::cout << "No memory remain.\n";
    }

    return 0;
}
```

Image 9

Analysis

Images1 and 2 show the code snippets for the object construction for the variables that the objects will carry such as the data types and their names. I used constructor and copy constructor methods to make them much easier and reusable. Image 3 shows the destructors for each of the constructed objects. Image 4 shows the function for the total sum and its formula along with the main function. Image 5 shows the main function for the receipts and its format. It also shows how items decrease when something is taken as long as it isn't pointing towards the null or 0. Images 6 and 7 basically just shows the process of deleting an item. Images 8 and 9 show the deletion process of the whole list and the deallocation process in order to remove all the memory consumed by the program.

Outputs

```
============================================================
                JENNA'S GROCERY RECEIPT
============================================================
Type            Item            Price     Quantity  Total
------------------------------------------------------------
Fruits:
Fruit           Apple           10        7         70
Fruit           Banana          10        8         80
Vegetables:
Vegetables      Broccoli        60        12        720
Vegetables      Lettuce         50        10        500

Total Amount in Pesos: 1370
```

Image 10

```
Proceed to Delete Lettuce? (Y/N): y

========== POINTER LIST BEFORE DELETION ==========
Apple: 0x7756d0
Banana: 0x77adf0
Broccoli: 0x7798b0
Lettuce: 0x779910
================================================

Deleting Lettuce...
Vegetable Destructor Called: Lettuce

Memory successfully deallocated.
```

Image 11

```
========== POINTER LIST AFTER DELETION ==========
Apple: 0x7756d0
Banana: 0x77adf0
Broccoli: 0x7798b0
Lettuce: 0
================================================

Lettuce successfully deleted.
Lettuce pointer is now nullptr.

Press Enter to Continue...
```

Image 12

```
============================================================
              UPDATED GROCERY RECEIPT
============================================================
Fruits:
Fruit          Apple          10        7         70
Fruit          Banana         10        8         80

Vegetables:
Vegetables     Broccoli       60        12        720

New Total Amount in Pesos: 870
```

Image 13

```
========== POINTER LIST BEFORE FINAL CLEANUP ==========
Apple: 0x7756d0
Banana: 0x77adf0
Broccoli: 0x7798b0
Lettuce: 0
============================================================

Cleaning up remaining memory
Fruit Destructor Called: Apple
Fruit Destructor Called: Banana
Vegetable Destructor Called: Broccoli

Memory cleaned up successfully.
```

Image 14

```
========== POINTER LIST AFTER FINAL CLEANUP ==========
Apple: 0
Banana: 0
Broccoli: 0
Lettuce: 0
============================================================

All allocated memory has been deallocated successfully.
No memory remain.
```

Image 15

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.